Terraform

- 🚩
- Intro
- Provider
- Resource Types
- Data Sources
- Module
- Query Resource & Data
  - Example: Deploy VPC and apps on it
- Init, Plan, Apply
  - 1. Init
  - 2. Plan
    - Planning Modes
    - Planning Options
  - Action Block
  - 3. Apply
- State
- Validate
- Dependency Graph
  - Types
- Destroy
- Remove
- Meta-Arguments
- Resource Reference & Expression
  - Sub-list w/ consecutive elem: slice(<list>, <startIdx>, <endIdx>)
  - Splat Expression
- Dynamic Block
- Output Block
  - 🛡️ ephemeral
- Variable Block
- Ephemeral Block
- Complex Types
  - Collection: list(type), map(type): same type
  - Structural: object(), tuple(): diff. types
- Built-in Function
  - Provider-def Function
- Validation
  - Variable: vide Variable Block
  - Precondition & Postcondition
  - Check Block
  - Validation Order
- Backend Block
- State Locking
- Refactor State
  - When?
  - How?
  - Migration
    - 1. Remove & Import ✅
      - Src Config Dir
      - Dest Config Dir
    - 2. Move resources directly to a new state file
- Moved Block
- Logging

- Tutorial 4a: Query AZs
- Tutorial 4f: DB Major Upgrade
- Tutorial 6d: Manage Resource Drift to Match Reality
- Tips: Round-Robin EC2 inst across private subnets
- HCP Terraform
  - Link Dependent Workspaces
  - Permissions
    - Org Role Perm
    - Project Role Perm
    - Workspace Role Perm
    - HCP Terraform Teams
  - Groups
  - Policy
  - Health Assessment & Drift Detection
  - Variable Set
  - State Migration to HCP Terraform
- Appendix
  - Cost Estimate

🚩

> ❌ Never upload the following to version control.
>
> - *AWS creds*
> - *plan*
> - *state*
> - `.tfvars`

Intro

Declarative language: What over How, spec desired result.

`terraform.tfstate` : **state file** *qua* source of truth

Day 0	Day 1	Day 2
Plan & Design	Deploy & Provision	Ops & Optm

Provider

- Plug-in for Terraform ↔ Cloud comm. via API
- Each provider adds a set of *resource types / data sources* qui Terraform mng.
- Resource type is implemented by a provider.
- Provider repo @ Terraform Registry.

1. `required_providers` : Provider Requirements, ensure `terraform init` does not install newer provider ver qui are incompatible w/ config.

`terraform.tf`

```
terraform {
  required_providers {
    mycloud = {
      source = "<ns/type>" # src adr.
      # Or
      source = "<hostname/ns/type>"

```

```
      version = "~> 3.2"
        # 3.2 <= ver < 4.0, Do NOT use it if reusing across multi confi

      alias = "eu-1"
        # alternate provider config by ref <PROVIDER>.<ALIAS>
    }
  }
}
```

`mycloud` *qua* local name `<PROVIDER_NAME>` for the provider: `"<hostname/ns/type>"`

`"<hostname/ns/type>"`:

- `hostname`: default registry.terraform.io
- `ns`: provider (plugin) publish org

Local name conflicts: multi providers w/ the same local name in `terraform.required_providers` block.

> Sol: `ns-type`, e.g. `hashicorp-http`, `mycorp-http` for `hashicorp/http` and `mycorp/http` source. In `required_providers` block.

2. `provider`: Provider Config

- auth, region, provider-spec args
- Child module recv provider config from parent modules.

`main.tf`

```
provider "<PROVIDER_NAME>" {
  <provider_args>
  alias = <ALIAS>
  version = "~> 2.0"
}
```

3. `terraform init` and update Dependency lock file, match version w/ ver config in `required_providers`

Resource Types

`TYPE` spec in `required_providers.localname.source` (hostname/ns/type)

Infra obj, incl. virtual network, comp. instances, DNS records.

`main.tf`

```
resource "<TYPE>" "<LABEL>"{
# TYPE: check provider (plugin) doc, e.g. aws_instance
  ami = <ami>
  instance_type = "t2.micro"
}
```

Use `map(object({…}))` to create multiple resources of the same type.

Data Sources

Fetch data about a resource

`main.tf`

```
data "<TYPE>" "<LABEL>" {
  count = num
  depends_on = [list of ref]
  provider = <PROVIDER_NAME>.<ALIAS>
}
```

Module

Split out groups of Terraform configs into reusable abstractions.

Path: `./modules/<module_name>`

Each module incl.: `main.tf`, `variables.tf`, `outputs.tf`

```
resource "<TYPE>" "<LABEL>" {}
data "<TYPE>" "<LABE>" {}
locals {}
```

Root module: `main.tf`

```
# Call a module
module "<LABEL>" {
  source = "./modules/<module_name>"
  # module args, vide resource, data block in the child module
  ...
}
```

Using Modules from Terraform Registry

```
module "<LABEL>" {
  source  = "<ns/name/provider>"
  # Or
  source  = "<hostname/ns/name/provider>"
  version = "<ver_constraint>"
}
```

Naming module `ns` for publishing: `terraform-<provider>-<name>`, e.g. `terraform-aws-modules`

`module.<LABEL>.<output_label>`: Ref a module output block.

Add `//` to source path: rest of the path after `//` is sub-dir w/in the package.

```
source = "hashicorp/consul/aws//modules/consul-cluster"
source = "git::https://example.com/network.git//modules/vpc"
```

Query Resource & Data

`<resource_type>.<label>[inst_idx]`, `data.<type>.<label>`, `module.<module_name>.[module_idx]`

Example: Deploy VPC and apps on it

`vpc/main.tf`

```
data "aws_availability_zones" "available" {
  state = "available"

  filter {
    name   = "zone-type"
    values = ["availability-zone"]
```

```
  }
}

module "vpc" {
  azs = data.aws_availability_zones.available.names
}
```

`app/main.tf`

```
# Ref to VPC state file
data "terraform_remote_state" "vpc" {
  backend = "local"

  config = {
    path = "../vpc/terraform.tfstate"
  }
}

provider "aws" {
  region = data.terraform_remote_state.vpc.outputs.aws_region
}

module "elb_http" {
  security_groups = data.terraform_remote_state.vpc.outputs.lb_security
  subnets         = data.terraform_remote_state.vpc.outputs.public_subn
}

resource "aws_instance" "app" {
  count = var.instances_per_subnet * length(data.terraform_remote_state
}
```

Init, Plan, Apply

1. Init

- If Provider or Module *ver* change → `terraform init -upgrade`
- If `backend`, `cloud` block change w/in `terraform` block → `terraform init`

Static check: `terraform validate` w/o running `terraform plan` or `terraform apply`:

```
terraform init -backend=false
```

2. Plan

1. Read cur state.
2. Compare cur config to prior state.
3. Propose a set of change actions.

```
terraform plan -out="<file>" -input=false # save gen-ed plan
terraform show -json "<file>" | jq > tfplan.json

terraform apply -input=false <file> # apply the plan
# -input=false: do not attempt to prompt for input
```

Planning Modes

- `terraform plan -destroy`: speculative plan for destruction.
- `terraform plan -refresh-only`: update state file to ***match reality***.
  - Scenario: change infra outside usual workflow, e.g. incident response.
  - Need to reconcile terraform records w/ the changes afterwards.

Planning Options

`invoke=action.<TYPE>.<LABEL>` : run a specific action def in config, *vide* Action

`-replace=<adr>` : force recreate, only for stateless resources

`-target=<adr>` : only look at spec. resource, ignore changes to everything else. ⚠️ Do NOT use in daily op, only when necessary.

Example: Emergency DNS fix `aws_route53_record.www_primary`

```
# Modify config
terraform plan -target="aws_route53_record.www_primary"
terraform apply -target="aws_route53_record.www_primary"
terraform plan
```

`-var '<name>=<val>'`, `-var-file=<file>`

N.B. no space before & after `=`

Modes & Options are shared b/w `plan` & `apply`

> ❌ Do **NOT** commit *plan files* to version control, whether binary or JSON format.

Action Block

Spec. provider-def. action to invoke w/ CLI or @ `apply`. e.g. Ansible playbooks, Lambda jobs.

```
action "aws_lambda_invoke" "example" {
  config {
    function_name = "func1" # refer function_name by name or ARN
    payload = jsonencode({
      k1 = "v1"
      k2 = "v2"
    })
  }
}
```

3. Apply

Idempotence: property of an operation or function that produces the same result or state, regardless of #times it exec.

State

1 remote object maps to 1 resource instance

```
terraform state # mod current state, e.g. removing items.
terraform state list # list deployment
terraform show # detail
terraform show "<planfile>" # detail of state or plan file

# Move resources from one state file to another
terraform state mv -state-out=../terraform.tfstate \
<emigrant-resource-type>.<resource-label> <immigrant-resource-type>.<re

# Remove resource from state file so terraform no longer mng
terraform state rm <resource>
```

State encryption 🔐: Remote backend (HCP Terraform), S3 `encrypt` option.

Validate

```
terraform init -backend=false
terraform validate -json
```

Dependency Graph

DAG (Direct Acyclic Graph), contains no cycle

Depender → Dependee (Depender relies on Dependee)

Types

- Resource Node
- Provider Config Node: time to fully config a provider.
- Resource Meta-Node: group of resources, does not represent any action.

Traversal done in parallel, Depth-first. Default: 10 nodes in the graph will be processed, num can be set w/ `-parallelism=<num>` flag on `plan`, `apply`, `destroy`.

Destroy

Deprovision all obj mng by Terraform config

```
terraform destroy
terraform apply -destroy # equivalence
```

Remove

Remove resources

`main.tf`

```
removed {
  from = aws_instance.example_new

  lifecycle {
    destroy = false # remove from tfstate file, but actual still exists.
  }
}
```

Meta-Arguments

`providers`: when multi `provider` are config-ed, use `providers` to specify provider, use `<PROVIDER>.<ALIAS>`

`main.tf`

```
provider "aws" {
  region = "us-west-1"
}

provider "aws" {
  alias  = "usw2"
  region = "us-west-2"
}

module "example" {
  source    = "./example"
  providers = {
    aws = aws.usw2
  }
}
```

`depends_on`: set order in which Terraform creates resources.

`lifecycle`: `destroy`, `precondition`, `postcondition`

`count`: `<TYPE>.<LABEL>[<INDEX>]`

- `count.index` : ref to inst.

```
locals {
  instance_names = ["web", "api", "batch"]
}

module "ec2_instance" {
  source  = "terraform-aws-modules/ec2-instance/aws"
  version = "6.0.2"
  count   = length(local.instance_names)

  name           = local.instance_names[count.index]
  ami            = data.aws_ami.latest_amazon_linux.id
  instance_type  = "t2.micro"

  depends_on = [aws_s3_bucket.example]
}
```

`for_each` : `<TYPE>.<LABEL>[<INDEX>]`

- `each.key` : map key or set member
- `each.value` : map value

`for_each` w/ Sets

```
locals {
  subnet_ids = toset([
    "subnet-123",
    "subnet-456",
  ])
}

resource "aws_instance" "server" {
  for_each = local.subnet_ids
  ...
  subnet_id = each.key

  tags = {
    Name = "Server ${each.key}"
  }
}

# Alt: config w/ input var
variable "subnet_ids" {
  type = set(string)
}
resource "aws_instance" "server" {
  for_each = var.subnet_ids
}
```

`for_each` w/ Maps

```
locals {
  instance_configs = {
    "inst-1" = { instance_type = "t3.nano" }
    "inst-2" = { instance_type = "t3.micro" }
    "inst-3" = { instance_tpye = "t3.small" }
  }
}

module "ec2_instance" {
  ...
  for_each = local.instance_configs

  name          = each.key
  instance_type = each.value.instance_type
}
```

Chain `for_each` b/w resources:

```
variable "vpcs" {
  type = map(object({
    cidr_block = string
```

```
  }))
}

resource "aws_vpc" "example" {
  # 1 VPC for each elem. of var.vpcs
  for_each = var.vpcs
  cidr_block = each.value.cidr_block
}

resource "aws_internet_gateway" "example" {
  # 1 Internet GW per VPC
  for_each = aws_vpc.example

  # each.value = aws_vpc object
  vpc_id = each.value.id
}

output "vpc_ids" {
  value = {
    for k, v in aws_vpc.example : k => v.id
  }

  # VPCs aren't fully functional until their
  # internet GW are running.
  depends_on = [aws_internet_gateway.example]
}
```

Resource Reference & Expression

```
# Cond. stmt
resource "random_id" "id" {
  byte_length = 8
}
locals {
  name  = (var.name != "" ? var.name : random_id.id.hex)
}

# Splat expr
output "tags" {
  description = "Instance tags for instance"
  value       = aws_instance.ubuntu[*].tags # or [0]
}
```

Sub-list w/ consecutive elem: `slice(<list>, <startIdx>, <endIdx>)`

- `startIdx` is inclusive; `endIdx` is exclusive
- e.g. `slice(["a", "b", "c", "d"], 1, 3)` → `["b", "c"]`

Splat Expression

```
var.list[*].id # Equiv: [for o in var.list : o.id]
```

Special behaviour when applying to types that is not list or set.

- If `null` → `[]` (empty list)
- Single elem. list

Dynamic Block

Construct repeatable nested block, inside `resource`, `data`, `provider`, `provisioner` blocks.

Usually def. var w/ `list( object({…}) )`

```
variable "settings" {
  type = list(object({
    # Def key / attr
    description = string
    port        = number
    protocol    = string
  }))
  default = [
    {
```

```
      description = "Allows SSH access"
      port        = 22
      protocol    = "tcp"
    },
    {
      description = "Allows HTTP traffic"
      port        = 80
      protocol    = "tcp"
    },
    {
      description = "Allows HTTPS traffic"
      port        = 443
      protocol    = "tcp"
    }
  ]
}

dynamic "<label>" { # label = arg w/in the parent block
  for_each = var.settings
  content {
    # <label>.value["<attr>"]
    description = <label>.value["description"]
    port = <label>.value["port"]
    protocol = <label>.value["protocol"]
  }
}
```

Output Block

```
output "<label>" {
  value     = <expr>
  sensitive = <bool>
  ephemeral = <bool>
}
```

`sensitive` : if `true` → output hidden in `plan` and `apply`.

- Plain text will be shown if:

```
terraform output -json # json opt allows all, or spec 1 output
terraform output -raw <output> # raw opt only allows 1 output

# Spec. the label
terraform output <output_label>
```

🛡️ `ephemeral`

- ***Not stored in state or plan files.***
- Set arg in `variable` and `output` blocks.
- Add in child module outputs.
- Root module
  - Not allowed: `output`
  - Allowed:
    - `provider`
    - write-only args (`password_wo`) in `resource` blocks
    - `locals` blocks.
- Scenario: cred, tk mgmt.

Get & set ephemeral var w/ spec. contexts

- `output` block w/ `ephemeral` arg: serves as getter func. for root module.

- Another `variable` block w/ `ephemeral` arg.
- Write-only arg: `_wo` & `_wo_version` in `resource` block.
- `ephemeral` block.
- `provider` block.

`modules/auth_tk/main.tf`

```
ephemeral "aws_ecr_authorization_token" "token" {
  registry_id = "123456789000"
}
```

`modules/auth_tk/outputs.tf`

```
output "proxy_password" {
  description = "Temp password"
  value       = ephemeral.aws_ecr_authorization_token.token.password
  # REQUIRED: Cannot output ephemeral val w/o arg: ephemeral = true
  ephemeral   = true
}
```

Root module `main.tf`

```
module "auth_tk" {
  source = "./modules/auth_tk"
}

provider "docker" {
  registry_auth {
    address  = "123456789000.dkr.ecr.us-east-1.amazonaws.com"
    username = "AWS"
    password = module.auth_tk.proxy_password # Only w/ output label
  }
}
```

Variable Block

Decl. in the block; Init in `.tfvars` or env var: `TF_VAR_<var_name>`

```
variable "image_id" {
  type        = string
  description = "AMI ID"

  validation {
    condition     = length(var.image_id) > 4 && substr(var.image_id, 0,
    error_message = "image_id value must be a valid AMI ID, starting w/
  }
}

# Optional var
variable "opt" {
  type = object({
    a = string
    b = optional(string)
  })
}
```

Ephemeral var

```
variable "access_key" {
  description = "AWS access key id"
  type        = string
  ephemeral   = true
}

variable "secret_key" {
  description = "AWS sensitive secret key"
  type        = string
```

```
  sensitive   = true
  ephemeral   = true
}

variable "session_token" {
  description = "AWS session token"
  type        = string
  sensitive   = true
  ephemeral   = true
}

provider "aws" {
  access_key = var.access_key
  secret_key = var.secret_key
  token      = var.session_token
}
```

Ephemeral Block

```
terraform { required_providers { random = { source = "hashicorp/random"

ephemeral "random_password" "db_password" {
  length  = 16
  special = false
}

resource "aws_db_instance" "foo" {
  ...
  password_wo         = ephemeral.random_password.db_password.result
  password_wo_version = local.db_password_version
}
```

Complex Types

Collection: `list(type)`, `map(type)` **: same type**

Structural: `object()`, `tuple()` **: diff. types**

```
object({
  <KEY> = <TYPE>,
  <KEY> = <TYPE>,
  ...
}

tuple([string, number, bool])
```

Built-in Function

Incl. numeric, string, collection, en/de-code, file sys, date time, type conversion

```
# Numeric
abs(), max(), min()

# String
split()
format("Int: %d, Float: %.2f, Str: %s", 20, 3.1415, "Hello")

# Collection
lookup(<map>, <key>) # get val from a map, given its key as arg
sort(<list>)

# File sys
templatefile("config.sh", { name = var.<var>, age = var.<var> })
file("<filename>")

# en/de-code
base64encode(), jsonencode()

# Datetime
timestamp() # now, @exec
plantimestamp() # spec planning time, consistent across resources
timeadd(ts, dur)
timecmp(ts_a, ts_b) # ts_a earlier: -1, same: 0, ts_a later: 1
```

Provider-def Function

```
provider::<PROVIDER_NAME>::func1()
```

Validation

Variable: vide [Variable Block]

Precondition & Postcondition

Verify resources, data sources, outputs

Postcondition: post-apply

`precondition` & `postcondition` blocks in `resource.lifecycle` / `data.lifecycle` blocks.

```
lifecycle {
  precondition {
    condition     = data.aws_ami.example.architecture == "x86_64"
    error_message = "Selected AMI must be for x86_64 arch."
  }
}
```

Check Block

- Post-apply validation
- Outside typical resource lifecycle
- Verify config w/o blocking operations.

```
check "health_check" {
  data "http" "terraform_io" {
    url = "https://www.terraform.io"
  }

  assert {
    condition = data.http.terraform_io.status_code == 200
    error_message = "${data.http.terraform_io.url} returns bad status c
  }
}
```

Continuous validation: check preconditions, postconditions, `check` blocks in HCP Terraform.

Validation Order

Validation	Phase
Variable	Init
Precondition	Plan
Postcondition	Apply
Check	Post-apply

Backend Block

Where Terraform stores state files.

- `cloud` and `backend` blocks cannot coexist.
- `backend` block *cannot* refer to named values (var, locals, data source attr)
- *Cannot* ref values decl. in `backend` blocks elsewhere in the config.

If use `-backend-config` or hardcode values directly in config, Terraform will incl. val. in both `.terraform` sub-dir and in plan files. This can leak sensitive creds.

Use env var to supply creds and other sensitive data.

```
backend "s3" {}

# Args: Env var
region: AWS_REGION
access_key: AWS_ACCESS_KEY_ID
secret_key: AWS_SECRET_ACCESS_KEY
profile: AWS_PROFILE
token: AWS_SESSION_TOKEN: MFA tk
https_proxy: HTTPS_PROXY: URL
iam: AWS_ENDPOINT_URL_IAM
s3: AWS_ENDPOINT_URL_S3
```

Env var `AWS_ENDPOINT_URL` can be used to set a base endpoint URL for all services.

Change backend requires migration, Terraform will ask.

State Locking

Terraform will lock state for all ops that could write state. It prevents others from acquiring the lock and potentially corrupting the state, acts as MUTEX proc. lock.

`terraform state push/pull`

`terraform state push` is extremely dangerous.

- **Differing lineage**: ID of a state, if different, likely modifying a different state, Terraform will not allow this.
- **Higher serial**: Every state has a mono. incr. serial nr (SN).
  - If dest. state SN > cur. state SN, Terraform will not allow to write.

Refactor State

When?

- Long applies: large config.
- Changes to mgmt lifecycles: separate resources per update frequency.
- Changes to resource ownership: split responsibility to diff. teams.
- Reusable modules

How?

- Volatility & rate of change: #comp resources vs network config
- Stateful vs stateless: compute inst vs DB
- Acc & team responsibility

Dependencies: dynamic ref., e.g. `aws_vpc`

Migration

1. Remove & Import ✅

Obj: `resource “aws_instance” “example” {}`

Src Config Dir

1. Backup: `terraform state pull > terraform.tfstate.backup`
2. Check required attire for import: `terraform state show <resource-type>.<resource-label>`
3. Delete `resource` block and add `removed` block:

```
removed {
  from = <resource-type>.<resource-label>
  ...
}
```

4. Ensure resources will not be destroyed: `plan`
5. Remove the resource from state: `apply`

Dest Config Dir

1. Add `resource` block
2. For each added resource, add `import` block

```
import {
  id = "<id>"
  to = aws_instance.example
}
```

3. Ensure proper import: `plan`
4. `apply`

2. Move resources directly to a new state file

⚠ `-state` and `-state-out` flags have risk of corrupting remote state.

1. Src config dir: `terraform state pull > source.tfstate`
2. Dest config dir: `terraform state pull > destination.tfstate`
3. Move state, vide State

```
terraform state mv \
-state source/source.tfstate \
-state-out destination/destination.tfstate \
<emigrant-resource> <immigrant-resouce>
```

4. Push back to backend

```
# Src config
terraform state push source.tfstate
# Dest config
terraform state push destination.tfstate
```

Moved Block

- For refactoring, not for migration, *i.e.* only 1 state file.
- Applies to both stateful and stateless resources.

Dest. config

```
moved {
  from = <adr>
  to   = <adr>
}
```

N.B. `removed`, `import`, `moved` **blocks should be deleted after running** `apply`**.**

Logging

```
export TF_LOG_CORE=TRACE
export TF_LOG_PATH=logs.txt
```

Tutorial 4a: Query AZs

Query data sources | Terraform | HashiCorp Developer

```
data "aws_availability_zones" "available" {
  state = "available"

  filter {
    name   = "zone-type"
    values = ["availability-zone"]
  }
}

module "vpc" {
  azs = data.aws_availability_zones.available.names
}
```

Tutorial 4f: DB Major Upgrade

Upgrade RDS major version | Terraform | HashiCorp Developer

1. Take snapshot → apply

```
resource "aws_db_snapshot" "pre_16_upgrade" {
  db_instance_identifier = aws_db_instance.education.identifier
  db_snapshot_identifier = "pre16upgradebackup"
}

output "rds_pre_16_backup_identifier" {
  description = "Identifier of the snapshot created before upgrading RD
  value = aws_db_snapshot.pre_16_upgrade.db_snapshot_identifier
}

output "rds_pre_16_backup_status" {
  description = "Status of the snapshot created before upgrading RDS in
  value = aws_db_snapshot.pre_16_upgrade.status
}
```

2. Upgrade DB: mod ver args → apply

Get ephemeral password

```
export PGPASSWORD=$( \
  aws ssm get-parameter \
    --region=$(terraform output -raw region) \
    --name=<aws_ssm_parameter.secret.name> \ # In resource "aws_ssm_para
    --with-decryption \
  | jq --raw-output '.Parameter.Value' \
  )
```

Tutorial 6d: Manage Resource Drift to Match Reality

```
terraform plan -refresh-only # update state file to match reality
terraform apply -refresh-only

# Update config file
resource "aws_instance" "example" {
  vpc_security_group_ids = [
    aws_security_group.sg_ssh.id,
    aws_security_group.sg_web.id # + sg_web
  ]
  ...
}

```

```
import {
  id = "<hardcoded-id>"
  to = aws_security_group.sg_web
}

import {
  id = "<hardcoded_id>"
  to = aws_security_group_rule.sg_web
}

terraform plan # import blocks still will not update state file
terraform apply
terraform state list
```

Tips: Round-Robin EC2 inst across private subnets

```
resource "aws_instance" "app" {
  ...
  subnet_id = var.aws_private_subnet_ids[
    count.index % length(var.aws_private_subnet_ids)
  ]
}
```

HCP Terraform

Workflow: VCS, CLI, API

- VCS: use webhook
- CLI: cmd → HCP Terraform → logs stream back to local terminal.

Agent

- Comm. w/ isolated, private, on-prem infra.
- Polls changes from HCP Terraform or Enterprise, no need public ingress to resources.
- Hooks: running custom programs at specific points.

Workspace: equiv to Working Directory run Terraform locally.

Project: group of workspaces, share variable sets.

Link Dependent Workspaces

`tfe_outputs`: acc. outputs from another workspace

```
data "tfe_outputs" "src" {
  workspace    = var.workspace_name
  organization = var.organization_name
}
resource "aws_instance" "app" {
    vpc_security_group_ids = data.tfe_outputs.src.nonsensitive_values.in
    subnet_id              = data.tfe_outputs.src.nonsensitive_values.in
}
```

Alt: [terraform_remote_state]() ref to state file.

`TFE_TOKEN`: share outputs inter-workspace.

1. Settings → API tokens → Team Tokens → Create a team token: Owners, 30 days (default)
   a. *Alt*: Upstream (dependee) workspace → Settings → General → Remote state sharing
2. Downstream (depender) workspace → Variables → Add variable → Environment variable: `TFE_TOKEN`, sensitive.

Run triggers: upstream workspace `apply` completion will trigger downstream workspace `apply`.

- Downstream workspace → Settings → Run Triggers → Connect workspace → Upstream Workspace: Connect.

💣 Destruction and Deletion: Workspace → Settings → Destruction and Deletion → Queue destroy plan: destroy all infra → Delete from HCP Terraform: delete workspace w/o destroy infra.

Permissions

Scope: Project, Workspace, Team, Settings, Private registry, Public registry

Org > Project > Workspace, *i.e.* perm set a a greater scale will override perm in a lesser scale.

Org Role Perm

Viewer: View all projects, View all workspaces.

Contributor: + View & Mng all projects, View & Mng all workspaces, Members can mng API tk.

Admin: All

Project Role Perm

Read: Read project, runs, var, states, outputs only.

Write: + `plan`, `apply`, R/W var, state, download Sentinel mocks, (un)lock workspace.

Maintain: + Create/Delete workspaces

Admin: All

Workspace Role Perm

Read: Read runs, var, outputs only, state.

Plan: + `plan` runs

Write: + `apply`, R/W var, state, download Sentinel mocks, (un)lock workspace.

Admin: All

HCP Terraform Teams

Owners Team: cannot delete or empty owners team.

Groups

- Separate IAM and HCP Terraform
- Past: Teams w/in HCP Terraform
- Now: HCP Portal: IAM, Group mgmt; HCP Terraform: Project, Workspace mgmt.
- For Owners:
  a. HCP Portal: create Owners Group, add users.
  b. HCP Terraform: Org Settings → Role Assignments (Users & Groups) → Assign Roles → Group: Role: Organization Owner

Policy

Rules for Terraform runs (`plan` , `apply` , `refresh-only` , `destroy`) to validate *qui* plans comply w/ security rules & best practices, e.g. deploy to correct region, prevent new infra deploy on Fridays.

Frameworks: Sentinel, OPA (Open Policy Agent)

Scope: Global, Project, Workspace

`policies.hcl` : Declare

```
policy "friday_deploys" {
  source = "./policies/<policy>.sentinel" # Sentinel framework
  query = "data.terraform.policies.friday_deploys.deny"
  params = {
    forbidden_days = ["friday"]
  }

  enforcement_level = "mandatory"
}
```

`enforcement_level`

Sentinel

- hard-mandatory: if fail → run stops.
- soft-mandatory: org owner or user w/ override perm proceed even if fail.
- advisory: if fail → proceed & notify.

OPA: mandatory, advisory. Soft/Hard def. in HCP Terraform Policy Set setting.

`./policies/friday_deploys.rego`

```
package terraform.policies.friday_deploys

deny[msg] {
  time.weekday(time.now_ns()) == "DAY"
  msg := "No deployments allowed on Fridays"
}
```

Health Assessment & Drift Detection

- Workspace → Health → Settings → Enable → Save settings
- Compare actual infra against state file, will not update state or infra config.

Variable Set

Lexical order if var sets asso. w/ the workspace contain the same var. Var sets → workspace: N → 1, >=2 sets have the same var def.

State Migration to HCP Terraform

- CLI: `cloud` block in `terraform` block → `terraform init`
- API:
  - `data.attributes.state` : base64 encoded `terraform.tfstate`
  - `data.attributes.md5` : `md5sum terraform.tfstate`
- `tf-migrate` :
  a. `terraform login` : obtain API tk for HCP Terraform.
  b. Setup env var `TF_GIT_PAT_TOKEN` : GitHub Personal Acc. Tk.

c. `git commit` → `git push origin master`

d. `tf-migrate prepare -skip-dir="dir_to_skip"` → `tf-migrate execute`

- `-skip-dir` : spec. prev loc. to store state, e.g. ***s3 (*** `key` : ***path to the file / obj)***

Appendix

Cost Estimate

Estimate hr/mo cost for terraform config.

Default: Disable

Support: AWS, GCP, Azure

```
import "tfrun"
import "decimal"

delta_mo_cost = decimal.new(tfrun.cost_estimate.delta_monthly_cost)

if delta_mo_cost.greater_than(100) {
  print("This policy prevents a user from incr spedning by >$100/mo in
}
main = rule {
  delta_mo_cost.less_than_or_equals(100)
}
```